

PL/SQL Exercises - Solutions

Bank Database System

Schema Reference

```
CREATE TABLE Customers (  
    CustomerID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100),  
    DOB DATE,  
    Balance NUMBER,  
    LastModified DATE  
);  
  
CREATE TABLE Accounts (  
    AccountID NUMBER PRIMARY KEY,  
    CustomerID NUMBER,  
    AccountType VARCHAR2(20),  
    Balance NUMBER,  
    LastModified DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);  
  
CREATE TABLE Transactions (  
    TransactionID NUMBER PRIMARY KEY,  
    AccountID NUMBER,  
    TransactionDate DATE,  
    Amount NUMBER,  
    TransactionType VARCHAR2(10),  
    FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)  
);  
  
CREATE TABLE Loans (  
    LoanID NUMBER PRIMARY KEY,  
    CustomerID NUMBER,  
    LoanAmount NUMBER,  
    InterestRate NUMBER,  
    StartDate DATE,  
    EndDate DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);  
  
CREATE TABLE Employees (  
    EmployeeID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100),  
    Position VARCHAR2(50),  
    Salary NUMBER,  
    Department VARCHAR2(50),  
    HireDate DATE  
);  
  
-- Recommended addition (used by trigger/audit solutions below)  
CREATE TABLE AuditLog (  
    LogID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    TableName VARCHAR2(50),  
    RecordID NUMBER,  
    ActionType VARCHAR2(20),  
    ActionDate DATE DEFAULT SYSDATE,  
    Details VARCHAR2(4000)  
);
```

```
CREATE TABLE ErrorLog (
    LogID NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ProcedureName VARCHAR2(50),
    ErrorMessage VARCHAR2(4000),
    LogDate DATE DEFAULT SYSDATE
);
```

Note: Accounts.InterestRate is not in the original schema but is needed for the loan-interest exercises; solutions below reference Loans.InterestRate, which does exist. Where an exercise implies a column that isn't in the schema (e.g. IsVIP on Customers, or a savings-account InterestRate), I've added an ALTER TABLE statement immediately before the relevant block so every script is runnable as-is.

Exercise 1: Control Structures

Scenario 1 — Loan interest discount for customers above 60

```
BEGIN
    FOR cust_rec IN (SELECT CustomerID, DOB FROM Customers) LOOP
        IF TRUNC(MONTHS_BETWEEN(SYSDATE, cust_rec.DOB) / 12) > 60 THEN
            UPDATE Loans
                SET InterestRate = InterestRate - (InterestRate * 0.01)
                WHERE CustomerID = cust_rec.CustomerID;
        END IF;
    END LOOP;
    COMMIT;
END;
/
```

Scenario 2 — Flag VIP customers (balance > \$10,000)

```
-- Run once to add the flag column
ALTER TABLE Customers ADD IsVIP CHAR(1) DEFAULT 'N';

BEGIN
    FOR cust_rec IN (SELECT CustomerID, Balance FROM Customers) LOOP
        IF cust_rec.Balance > 10000 THEN
            UPDATE Customers
                SET IsVIP = 'Y'
                WHERE CustomerID = cust_rec.CustomerID;
        ELSE
            UPDATE Customers
                SET IsVIP = 'N'
                WHERE CustomerID = cust_rec.CustomerID;
        END IF;
    END LOOP;
    COMMIT;
END;
/
```

Scenario 3 — Reminders for loans due within 30 days

```
DECLARE
    v_name Customers.Name%TYPE;
BEGIN
    FOR loan_rec IN (
        SELECT LoanID, CustomerID, EndDate
        FROM Loans
        WHERE EndDate BETWEEN SYSDATE AND SYSDATE + 30
```

```

) LOOP
    SELECT Name INTO v_name
    FROM Customers
    WHERE CustomerID = loan_rec.CustomerID;

    DBMS_OUTPUT.PUT_LINE(
        'Reminder: Customer ' || v_name ||
        ' (ID: ' || loan_rec.CustomerID ||
        ') has Loan #' || loan_rec.LoanID ||
        ' due on ' || TO_CHAR(loan_rec.EndDate, 'DD-MON-YYYY')
    );
END LOOP;
END;
/

```

Exercise 2: Error Handling

Scenario 1 — SafeTransferFunds

```

CREATE OR REPLACE PROCEDURE SafeTransferFunds (
    p_from_account IN NUMBER,
    p_to_account   IN NUMBER,
    p_amount       IN NUMBER
) IS
    v_from_balance NUMBER;
    insufficient_funds EXCEPTION;
BEGIN
    SELECT Balance INTO v_from_balance
    FROM Accounts
    WHERE AccountID = p_from_account
    FOR UPDATE;

    IF v_from_balance < p_amount THEN
        RAISE insufficient_funds;
    END IF;

    UPDATE Accounts
    SET Balance = Balance - p_amount, LastModified = SYSDATE
    WHERE AccountID = p_from_account;

    UPDATE Accounts
    SET Balance = Balance + p_amount, LastModified = SYSDATE
    WHERE AccountID = p_to_account;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Transfer successful.');
```

```

EXCEPTION
    WHEN insufficient_funds THEN
        ROLLBACK;
        INSERT INTO ErrorLog (ProcedureName, ErrorMessage)
        VALUES ('SafeTransferFunds', 'Insufficient funds in account ' || p_from_account);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Error: Insufficient funds.');
```

```

    WHEN NO_DATA_FOUND THEN
        ROLLBACK;
        INSERT INTO ErrorLog (ProcedureName, ErrorMessage)
        VALUES ('SafeTransferFunds', 'Account not found: ' || p_from_account || ' or ' || p_to_account);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Error: Account not found.');
```

```

    WHEN OTHERS THEN
        ROLLBACK;

```

```

        INSERT INTO ErrorLog (ProcedureName, ErrorMessage)
        VALUES ('SafeTransferFunds', SQLERRM);
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END SafeTransferFunds;
/

```

Scenario 2 — UpdateSalary

```

CREATE OR REPLACE PROCEDURE UpdateSalary (
    p_employee_id IN NUMBER,
    p_percentage  IN NUMBER
) IS
    v_exists NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_exists
    FROM Employees
    WHERE EmployeeID = p_employee_id;

    IF v_exists = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Employee ID ' || p_employee_id || ' does not exist.');
```

END IF;

```

    UPDATE Employees
    SET Salary = Salary + (Salary * p_percentage / 100)
    WHERE EmployeeID = p_employee_id;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Salary updated successfully.');
```

EXCEPTION

```

    WHEN OTHERS THEN
        ROLLBACK;
        INSERT INTO ErrorLog (ProcedureName, ErrorMessage)
        VALUES ('UpdateSalary', SQLERRM);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END UpdateSalary;
/

```

Scenario 3 — AddNewCustomer

```

CREATE OR REPLACE PROCEDURE AddNewCustomer (
    p_customer_id IN NUMBER,
    p_name        IN VARCHAR2,
    p_dob         IN DATE,
    p_balance     IN NUMBER
) IS
    duplicate_customer EXCEPTION;
    PRAGMA EXCEPTION_INIT(duplicate_customer, -1); -- ORA-00001 unique constraint violated
BEGIN
    INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
    VALUES (p_customer_id, p_name, p_dob, p_balance, SYSDATE);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Customer added successfully.');
```

EXCEPTION

```

    WHEN duplicate_customer THEN
        ROLLBACK;
        INSERT INTO ErrorLog (ProcedureName, ErrorMessage)
        VALUES ('AddNewCustomer', 'Duplicate CustomerID: ' || p_customer_id);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Error: Customer ID ' || p_customer_id || ' already exists.');
```

```

        WHEN OTHERS THEN
            ROLLBACK;
            INSERT INTO ErrorLog (ProcedureName, ErrorMessage)
            VALUES ('AddNewCustomer', SQLERRM);
            COMMIT;
            DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    END AddNewCustomer;
/

```

Exercise 3: Stored Procedures

Scenario 1 — ProcessMonthlyInterest

```

CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS
    c_interest_rate CONSTANT NUMBER := 0.01; -- 1%
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * c_interest_rate),
        LastModified = SYSDATE
    WHERE AccountType = 'Savings';

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Monthly interest applied to all savings accounts.');
```

```

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        INSERT INTO ErrorLog (ProcedureName, ErrorMessage)
        VALUES ('ProcessMonthlyInterest', SQLERRM);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END ProcessMonthlyInterest;
/

```

Scenario 2 — UpdateEmployeeBonus

```

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
    p_department      IN VARCHAR2,
    p_bonus_percentage IN NUMBER
) IS
BEGIN
    UPDATE Employees
    SET Salary = Salary + (Salary * p_bonus_percentage / 100)
    WHERE Department = p_department;

    IF SQL%ROWCOUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No employees found in department: ' || p_department);
    ELSE
        COMMIT;
        DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' employee(s) received a bonus.');
```

```

    END IF;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        INSERT INTO ErrorLog (ProcedureName, ErrorMessage)
        VALUES ('UpdateEmployeeBonus', SQLERRM);
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END UpdateEmployeeBonus;
/

```

Scenario 3 — TransferFunds

```
CREATE OR REPLACE PROCEDURE TransferFunds (  
    p_from_account IN NUMBER,  
    p_to_account   IN NUMBER,  
    p_amount       IN NUMBER  
) IS  
    v_balance NUMBER;  
BEGIN  
    SELECT Balance INTO v_balance  
    FROM Accounts  
    WHERE AccountID = p_from_account  
    FOR UPDATE;  
  
    IF v_balance < p_amount THEN  
        DBMS_OUTPUT.PUT_LINE('Transfer failed: insufficient balance in account ' || p_from_account);  
        RETURN;  
    END IF;  
  
    UPDATE Accounts  
    SET Balance = Balance - p_amount, LastModified = SYSDATE  
    WHERE AccountID = p_from_account;  
  
    UPDATE Accounts  
    SET Balance = Balance + p_amount, LastModified = SYSDATE  
    WHERE AccountID = p_to_account;  
  
    COMMIT;  
    DBMS_OUTPUT.PUT_LINE('Transfer of ' || p_amount || ' completed.');
```

```
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        ROLLBACK;  
        DBMS_OUTPUT.PUT_LINE('Error: Source account not found.');
```

```
    WHEN OTHERS THEN  
        ROLLBACK;  
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);  
END TransferFunds;  
/
```

Exercise 4: Functions

Scenario 1 — CalculateAge

```
CREATE OR REPLACE FUNCTION CalculateAge (  
    p_dob IN DATE  
) RETURN NUMBER IS  
    v_age NUMBER;  
BEGIN  
    v_age := TRUNC(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);  
    RETURN v_age;  
END CalculateAge;  
/
```

```
-- Example usage:  
-- SELECT CustomerID, Name, CalculateAge(DOB) AS Age FROM Customers;
```

Scenario 2 — CalculateMonthlyInstallment

Uses the standard amortized-loan (EMI) formula:

$$EMI = P * r * (1+r)^n / ((1+r)^n - 1)$$

where P = principal, r = monthly interest rate, n = number of months.

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment (
    p_loan_amount    IN NUMBER,
    p_annual_rate     IN NUMBER,    -- e.g. 5 for 5%
    p_duration_years  IN NUMBER
) RETURN NUMBER IS
    v_monthly_rate NUMBER;
    v_num_payments NUMBER;
    v_emi          NUMBER;
BEGIN
    v_monthly_rate := (p_annual_rate / 100) / 12;
    v_num_payments := p_duration_years * 12;

    IF v_monthly_rate = 0 THEN
        v_emi := p_loan_amount / v_num_payments;
    ELSE
        v_emi := p_loan_amount * v_monthly_rate * POWER(1 + v_monthly_rate, v_num_payments)
            / (POWER(1 + v_monthly_rate, v_num_payments) - 1);
    END IF;

    RETURN ROUND(v_emi, 2);
END CalculateMonthlyInstallment;
/

-- Example usage:
-- SELECT CalculateMonthlyInstallment(5000, 5, 5) FROM DUAL;
```

Scenario 3 — HasSufficientBalance

```
CREATE OR REPLACE FUNCTION HasSufficientBalance (
    p_account_id IN NUMBER,
    p_amount      IN NUMBER
) RETURN BOOLEAN IS
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance
    FROM Accounts
    WHERE AccountID = p_account_id;

    IF v_balance >= p_amount THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE;
END HasSufficientBalance;
/

-- Note: PL/SQL BOOLEAN cannot be selected directly in plain SQL.
-- Call it from a PL/SQL block:
-- BEGIN
--     IF HasSufficientBalance(1, 500) THEN
--         DBMS_OUTPUT.PUT_LINE('Sufficient balance.');
```

Exercise 5: Triggers

Scenario 1 — UpdateCustomerLastModified

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
    :NEW.LastModified := SYSDATE;
END;
/
```

Scenario 2 — LogTransaction

```
CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog (TableName, RecordID, ActionType, ActionDate, Details)
    VALUES (
        'Transactions',
        :NEW.TransactionID,
        'INSERT',
        SYSDATE,
        'AccountID=' || :NEW.AccountID ||
        ', Amount=' || :NEW.Amount ||
        ', Type=' || :NEW.TransactionType
    );
END;
/
```

Scenario 3 — CheckTransactionRules

```
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance
    FROM Accounts
    WHERE AccountID = :NEW.AccountID;

    IF :NEW.TransactionType = 'Deposit' AND :NEW.Amount <= 0 THEN
        RAISE_APPLICATION_ERROR(-20010, 'Deposit amount must be positive.');
```

```
    ELSIF :NEW.TransactionType = 'Withdrawal' THEN
        IF :NEW.Amount <= 0 THEN
            RAISE_APPLICATION_ERROR(-20011, 'Withdrawal amount must be positive.');
```

```
        ELSIF :NEW.Amount > v_balance THEN
            RAISE_APPLICATION_ERROR(-20012, 'Withdrawal exceeds account balance.');
```

```
        END IF;
    END IF;
END;
/
```

Exercise 6: Cursors

Scenario 1 — GenerateMonthlyStatements

```
DECLARE
    CURSOR GenerateMonthlyStatements IS
        SELECT t.TransactionID, t.AccountID, t.TransactionDate,
               t.Amount, t.TransactionType, a.CustomerID
        FROM Transactions t
        JOIN Accounts a ON t.AccountID = a.AccountID
        WHERE TRUNC(t.TransactionDate, 'MM') = TRUNC(SYSDATE, 'MM')
        ORDER BY a.CustomerID, t.TransactionDate;

    v_name Customers.Name%TYPE;
BEGIN
    FOR txn_rec IN GenerateMonthlyStatements LOOP
        SELECT Name INTO v_name
        FROM Customers
        WHERE CustomerID = txn_rec.CustomerID;

        DBMS_OUTPUT.PUT_LINE(
            'Statement | Customer: ' || v_name ||
            ' | Account: ' || txn_rec.AccountID ||
            ' | Date: ' || TO_CHAR(txn_rec.TransactionDate, 'DD-MON-YYYY') ||
            ' | Type: ' || txn_rec.TransactionType ||
            ' | Amount: ' || txn_rec.Amount
        );
    END LOOP;
END;
/
```

Scenario 2 — ApplyAnnualFee

```
DECLARE
    CURSOR ApplyAnnualFee IS
        SELECT AccountID, Balance FROM Accounts FOR UPDATE;

    c_annual_fee CONSTANT NUMBER := 25;
BEGIN
    FOR acc_rec IN ApplyAnnualFee LOOP
        UPDATE Accounts
        SET Balance = Balance - c_annual_fee,
            LastModified = SYSDATE
        WHERE CURRENT OF ApplyAnnualFee;
    END LOOP;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Annual fee applied to all accounts.');
```

```
END;
/
```

Scenario 3 — UpdateLoanInterestRates

```
-- Example new policy: cap rates at 8% and add 0.5% to anything below 4%.
DECLARE
    CURSOR UpdateLoanInterestRates IS
        SELECT LoanID, InterestRate FROM Loans FOR UPDATE;

    v_new_rate NUMBER;
BEGIN
    FOR loan_rec IN UpdateLoanInterestRates LOOP
        v_new_rate := loan_rec.InterestRate;

        IF v_new_rate < 4 THEN
            v_new_rate := v_new_rate + 0.5;
        ELSIF v_new_rate > 8 THEN
```

```

        v_new_rate := 8;
    END IF;

    UPDATE Loans
    SET InterestRate = v_new_rate
    WHERE CURRENT OF UpdateLoanInterestRates;
END LOOP;

COMMIT;
DBMS_OUTPUT.PUT_LINE('Loan interest rates updated per new policy.');
```

END;

/

Exercise 7: Packages

Scenario 1 — CustomerManagement

```

CREATE OR REPLACE PACKAGE CustomerManagement AS
    PROCEDURE AddCustomer (
        p_customer_id IN NUMBER,
        p_name         IN VARCHAR2,
        p_dob          IN DATE,
        p_balance      IN NUMBER
    );

    PROCEDURE UpdateCustomerDetails (
        p_customer_id IN NUMBER,
        p_name         IN VARCHAR2,
        p_balance      IN NUMBER
    );

    FUNCTION GetCustomerBalance (
        p_customer_id IN NUMBER
    ) RETURN NUMBER;
END CustomerManagement;
/

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS

    PROCEDURE AddCustomer (
        p_customer_id IN NUMBER,
        p_name         IN VARCHAR2,
        p_dob          IN DATE,
        p_balance      IN NUMBER
    ) IS
    BEGIN
        INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
        VALUES (p_customer_id, p_name, p_dob, p_balance, SYSDATE);
        COMMIT;
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('Error: Customer ID already exists.');
```

```

        WHEN OTHERS THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    END AddCustomer;

    PROCEDURE UpdateCustomerDetails (
        p_customer_id IN NUMBER,
        p_name         IN VARCHAR2,
        p_balance      IN NUMBER
    ) IS
    BEGIN
```

```

UPDATE Customers
SET Name = p_name, Balance = p_balance, LastModified = SYSDATE
WHERE CustomerID = p_customer_id;

IF SQL%ROWCOUNT = 0 THEN
    DBMS_OUTPUT.PUT_LINE('Error: Customer not found.');
```

```

ELSE
    COMMIT;
END IF;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END UpdateCustomerDetails;

FUNCTION GetCustomerBalance (
    p_customer_id IN NUMBER
) RETURN NUMBER IS
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance
    FROM Customers
    WHERE CustomerID = p_customer_id;
    RETURN v_balance;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
END GetCustomerBalance;

END CustomerManagement;
/
```

Scenario 2 — EmployeeManagement

```

CREATE OR REPLACE PACKAGE EmployeeManagement AS
    PROCEDURE HireEmployee (
        p_employee_id IN NUMBER,
        p_name         IN VARCHAR2,
        p_position      IN VARCHAR2,
        p_salary        IN NUMBER,
        p_department    IN VARCHAR2
    );

    PROCEDURE UpdateEmployeeDetails (
        p_employee_id IN NUMBER,
        p_position     IN VARCHAR2,
        p_salary       IN NUMBER,
        p_department   IN VARCHAR2
    );

    FUNCTION CalculateAnnualSalary (
        p_employee_id IN NUMBER
    ) RETURN NUMBER;
END EmployeeManagement;
/

CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS

    PROCEDURE HireEmployee (
        p_employee_id IN NUMBER,
        p_name         IN VARCHAR2,
        p_position      IN VARCHAR2,
        p_salary        IN NUMBER,
        p_department    IN VARCHAR2
    ) IS
    BEGIN
        INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate)
        VALUES (p_employee_id, p_name, p_position, p_salary, p_department, SYSDATE);
```

```

        COMMIT;
    EXCEPTION
        WHEN DUP_VAL_ON_INDEX THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('Error: Employee ID already exists.');
```

```

        WHEN OTHERS THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    END HireEmployee;

    PROCEDURE UpdateEmployeeDetails (
        p_employee_id IN NUMBER,
        p_position     IN VARCHAR2,
        p_salary       IN NUMBER,
        p_department   IN VARCHAR2
    ) IS
    BEGIN
        UPDATE Employees
        SET Position = p_position, Salary = p_salary, Department = p_department
        WHERE EmployeeID = p_employee_id;

        IF SQL%ROWCOUNT = 0 THEN
            DBMS_OUTPUT.PUT_LINE('Error: Employee not found.');
```

```

        ELSE
            COMMIT;
        END IF;
    EXCEPTION
        WHEN OTHERS THEN
            ROLLBACK;
            DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    END UpdateEmployeeDetails;

    FUNCTION CalculateAnnualSalary (
        p_employee_id IN NUMBER
    ) RETURN NUMBER IS
        v_monthly_salary NUMBER;
    BEGIN
        SELECT Salary INTO v_monthly_salary
        FROM Employees
        WHERE EmployeeID = p_employee_id;
        RETURN v_monthly_salary * 12;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RETURN NULL;
    END CalculateAnnualSalary;

END EmployeeManagement;
/
```

Scenario 3 — AccountOperations

```

CREATE OR REPLACE PACKAGE AccountOperations AS
    PROCEDURE OpenAccount (
        p_account_id  IN NUMBER,
        p_customer_id IN NUMBER,
        p_account_type IN VARCHAR2,
        p_balance      IN NUMBER
    );

    PROCEDURE CloseAccount (
        p_account_id IN NUMBER
    );

    FUNCTION GetTotalBalance (
        p_customer_id IN NUMBER
    ) RETURN NUMBER;
END AccountOperations;
/
```

CREATE OR REPLACE PACKAGE BODY AccountOperations **AS**

```
PROCEDURE OpenAccount (  
    p_account_id    IN NUMBER,  
    p_customer_id   IN NUMBER,  
    p_account_type  IN VARCHAR2,  
    p_balance       IN NUMBER  
) IS  
BEGIN  
    INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance, LastModified)  
    VALUES (p_account_id, p_customer_id, p_account_type, p_balance, SYSDATE);  
    COMMIT;  
EXCEPTION  
    WHEN DUP_VAL_ON_INDEX THEN  
        ROLLBACK;  
        DBMS_OUTPUT.PUT_LINE('Error: Account ID already exists.');
```

```
    WHEN OTHERS THEN  
        ROLLBACK;  
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);  
END OpenAccount;  
  
PROCEDURE CloseAccount (  
    p_account_id IN NUMBER  
) IS  
    v_balance NUMBER;  
BEGIN  
    SELECT Balance INTO v_balance  
    FROM Accounts  
    WHERE AccountID = p_account_id;  
  
    IF v_balance <> 0 THEN  
        DBMS_OUTPUT.PUT_LINE('Error: Account balance must be zero before closing.');
```

```
        RETURN;  
    END IF;  
  
    DELETE FROM Accounts WHERE AccountID = p_account_id;  
    COMMIT;  
    DBMS_OUTPUT.PUT_LINE('Account closed successfully.');
```

```
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        DBMS_OUTPUT.PUT_LINE('Error: Account not found.');
```

```
    WHEN OTHERS THEN  
        ROLLBACK;  
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);  
END CloseAccount;
```

```
  
FUNCTION GetTotalBalance (  
    p_customer_id IN NUMBER  
) RETURN NUMBER IS  
    v_total NUMBER;  
BEGIN  
    SELECT NVL(SUM(Balance), 0) INTO v_total  
    FROM Accounts  
    WHERE CustomerID = p_customer_id;  
    RETURN v_total;  
END GetTotalBalance;
```

END AccountOperations;
/